Day 130: Training a VAE, and a Real Latent-Space Visualisation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 19, Day 5 | Complete study guide

Watch: Variational Autoencoders simplified (Prof. Ryan Ahmed) | Variational Autoencoder - VISUALLY EXPLAINED! (Kapil Sachdeva)

Script: `day130_vae_latent_space.py` · every number in this guide is copied by program from `day130_run_output.txt`, the real output of that script.

Day 129's variational autoencoder, trained for 30 epochs on MNIST, against Day 126's plain autoencoder. It rebuilds almost as well (0.01236 against 0.01212 per pixel), and its codes can be sampled: images decoded from N(0, 1) sit much closer to real digits (0.1857) than the plain autoencoder's random codes (0.2048), near real images' own 0.1743. With two code numbers the whole digit space fits on one page. One warning, measured: the closeness score rewards blur, so heavier KL weighting looks 'better' (0.1695) while its images get vaguer.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

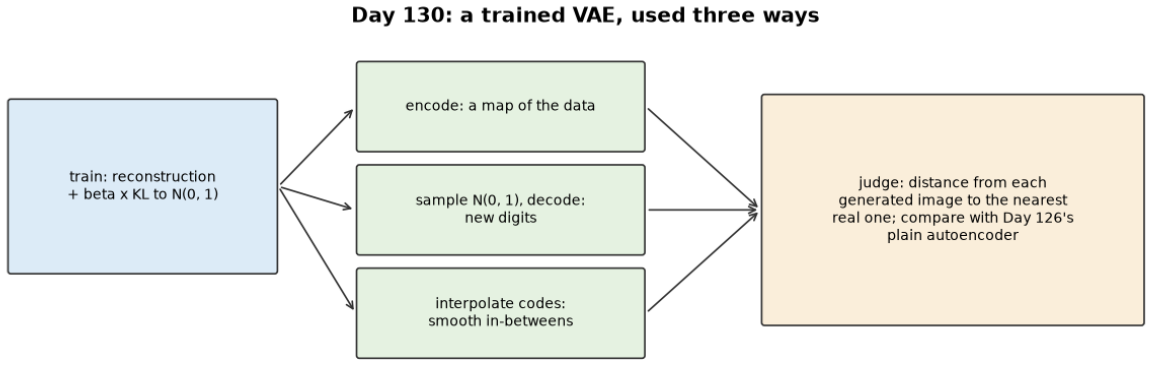
The **latent space** (code space) is where the codes live. A VAE's KL term pulls every image's code cloud towards N(0, 1), so the codes fill a known region with no large gaps. Then three things become possible: **sampling** (draw a code from N(0, 1), decode it), **interpolation** (walk between two codes) and **mapping** (with 2 code numbers, draw the whole space).

Beta (β) weights the KL term: larger β pulls codes harder towards N(0, 1), at the cost of reconstruction.

REAL-WORLD SCENARIO

A library where every book has a shelf position, and the shelves have no empty gaps: walk anywhere along them and you find a book, and neighbouring books are on similar subjects. A plain autoencoder's library has scattered shelves with empty corridors in between.

SYSTEM DIAGRAM



Train once; then encode, sample and interpolate; judge against Day 126's plain autoencoder.

Today's part for the week's build

`day130_exercise.py` is encoding, decoding, sampling, interpolation and the two judging tools, checked by hand. Its real output:

ACTUAL OUTPUT

```
encode: means (6, 3), log-variances (6, 3); sample_prior: (4, 10), all pixels in (0, 1): True
interpolate from 0 to (2, 4, 6) in 5 steps, first coordinate: [0.0, 0.5, 1.0, 1.5, 2.0] (by hand
    0, 0.5, 1, 1.5, 2)
nearest_real: an image in the reference set is 0.0000 from it; an all-0.5 image from an all-0
    reference: 0.5000 (by hand 0.5)
an encoder that always outputs N(0, 1): active dimensions 0, KL per dimension [0.0, 0.0, 0.0]
```

Data: MNIST (cached by Day 126), 20,000 training images; labels used only to colour and judge. Day 129's VAE parts and Day 126's network parts are imported unchanged.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. Training: a VAE against a plain autoencoder

DEFINITION — start here, no prior knowledge assumed

Both with 16 code numbers and the same 784-256 layers, 30 epochs on 20,000 images.

REAL-WORLD SCENARIO

Two filing systems for the same documents: one only has to find each document again; the other must also keep its shelves evenly filled.

IMPLEMENTATION

```
def encode(enc, X):
    H = d126.forward(enc, X, ENC)[-1]; k = H.shape[1] // 2
    return H[:, :k], H[:, k:]
def decode(dec, Z):
    return d126.sigmoid(d126.forward(dec, Z.astype(np.float32), DEC)[-1])
def sample_prior(dec, n, k, rng):
    return decode(dec, rng.standard_normal((n, k)))
```

ACTUAL OUTPUT

```
VAE: test loss 109.11 nats per image = reconstruction 85.27 + KL 23.84; trained in 140 s
error per pixel, rebuilding from the code mean: VAE 0.01236; plain autoencoder 0.01212 (trained in 141 s)
held-out loss every 10 epochs: [115.14, 110.86, 109.21]
the VAE rebuilds a little worse: the KL term spends some of its capacity keeping the codes near N(0, 1)
Actual output from this lesson's run.
```

What it means

The VAE's loss is 109.11 nats per test image: 85.27 for reconstruction and 23.84 for the KL term. Rebuilding from the code mean, its error per pixel (0.01236) is a little above the plain autoencoder's (0.01212): keeping codes near N(0, 1) costs some reconstruction. Held-out loss was still falling slowly ([115.14, 110.86, 109.21] at epochs 10, 20, 30).

WHY CHOOSE THIS?

Nearly the same reconstructions, with a usable code space.

WHY NOT THIS? (TRADEOFFS)

Slightly worse reconstruction, and one more term to balance.

2. By hand: one image's loss

DEFINITION — start here, no prior knowledge assumed

One test image's 16 code means and spreads, its KL summed by hand, and its reconstruction cost.

REAL-WORLD SCENARIO

Itemising one bill: the delivery and the storage, added up.

MATHEMATICS

$$loss = reconstruction + KL = -E_q[\log p(x|z)] + \tfrac{1}{2}\sum_j(\sigma_j^2 + \mu_j^2 - 1 - \log \sigma_j^2) = -ELBO$$

ACTUAL OUTPUT

```
its 16 code means range from -1.918 to 2.180; its spreads (sigma) from 0.167 to 0.359
KL by hand, summing 0.5 (sigma^2 + mu^2 - 1 - log sigma^2) over the 16 dimensions: 22.675 nats;
    vae_loss: 22.675
reconstruction (cross-entropy, one draw of z): 60.050 nats; loss = 60.050 + 22.675 = 82.725, the
    negative ELBO
Actual output from this lesson's run.
```

What it means

Its code means lie between -1.918 and 2.180, its spreads between 0.167 and 0.359: each image's cloud is much narrower than N(0, 1), while the KL term pushes the clouds of all images together to cover it. Summed by hand, the KL is 22.675 nats, matching vae_loss; with reconstruction 60.050, the loss is 82.725, below the test average, so this is an easier-than-average image.

WHY CHOOSE THIS?

Every term of the loss, checked on one image.

WHY NOT THIS? (TRADEOFFS)

One draw of z; the reconstruction term varies a little from draw to draw.

3. The two-dimensional code space

DEFINITION — start here, no prior knowledge assumed

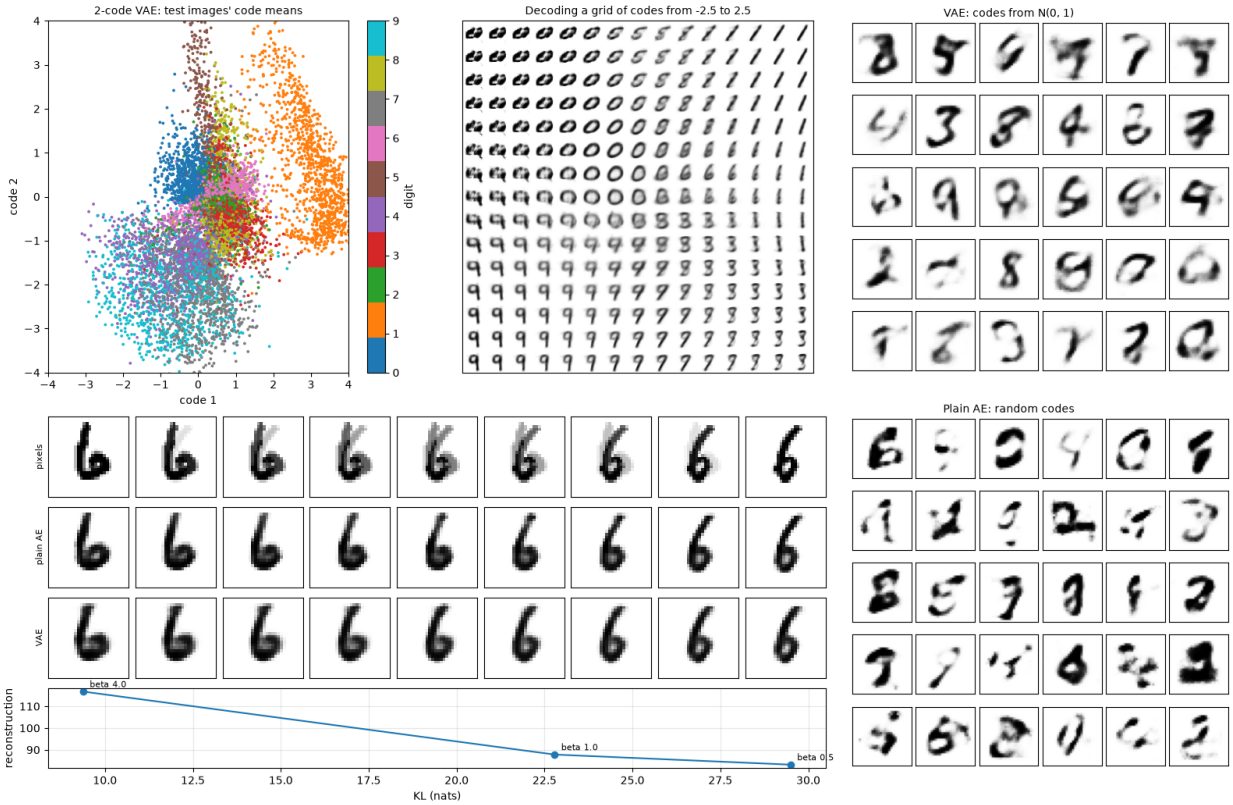
A VAE with only 2 code numbers, so every test image is a point on the page, and a grid of codes can be decoded.

REAL-WORLD SCENARIO

A map of every handwritten digit on one sheet of paper.

ACTUAL OUTPUT

```
test loss 156.96 nats (reconstruction 151.16, KL 5.80) with only 2 numbers per image
code means of the 10,000 test images: mean (0.492, -0.447), spread (1.180, 1.288); N(0, 1) would
    give (0, 0) and (1, 1)
kNN (k = 3) on the 2 code numbers, labels used only to judge: 0.5291
Actual output from this lesson's run.
```



Top: the 2-code VAE's map of test images, coloured by digit; decoding a 15 x 15 grid of codes; 30 images decoded from N(0, 1). Bottom: walking between two test images by pixels, plain autoencoder and VAE; reconstruction against KL for three betas; 30 images from the plain autoencoder's random codes.

What it means

Two numbers are not enough to rebuild well (151.16 nats, against 85.27 with 16), but they make a real map: in the figure each digit occupies a region, and the decoded grid changes smoothly from 0 through 6 and 8 to 1, and from 9 through 7 to 3. The codes roughly fill N(0, 1) (spread (1.180, 1.288) against 1), and a kNN on the 2 numbers scores 0.5291: neighbouring codes are often, not always, the same digit.

WHY CHOOSE THIS?

The whole data set, visible at once.

WHY NOT THIS? (TRADEOFFS)

Two dimensions squash many digits together (4, 9 and 7 overlap).

4. Generating digits

DEFINITION — start here, no prior knowledge assumed

1,000 codes drawn at random and decoded: from N(0, 1) for the VAE; with the real codes' mean and spread for the plain autoencoder (Day 126's method). Each image is compared with the nearest of 10,000 real images neither model trained on; 1,000 real images give the yardstick.

REAL-WORLD SCENARIO

Pulling books from random shelf positions, and asking how close each is to a real book.

IMPLEMENTATION

```
def nearest_real(R, ref, chunk=500):
    return np.concatenate([np.sqrt(np.maximum(d119.sqdist(R[s:s + chunk].astype(np.float64), ref),
        0).min(1) / R.shape[1])
                           for s in range(0, len(R), chunk)])
def active_units(enc, X, thresh=0.1):
    mu, lv = encode(enc, X); kl = (0.5 * (np.exp(lv) + mu ** 2 - 1 - lv)).mean(0)
    return int((kl > thresh).sum()), kl
```

ACTUAL OUTPUT

```
  real images (yardstick)          : distance to the nearest real image, per pixel: median 0.1743, 90th
      percentile 0.2222
  plain autoencoder, random codes  : distance to the nearest real image, per pixel: median 0.2048, 90th
      percentile 0.2486
  VAE, codes from N(0, 1)          : distance to the nearest real image, per pixel: median 0.1857, 90th
      percentile 0.2197
Actual output from this lesson's run.
```

What it means

VAE samples sit much closer to real digits (median 0.1857) than the plain autoencoder's (0.2048), near the real images' own 0.1743. In the figure, most VAE samples are recognisable digits, some blurred; most plain-autoencoder samples are blots. By this measure the KL term closed most of Day 126's gaps. A caution about the score: a blurry, average-looking image is close to many real ones, so this measure rewards blur (section 6 shows it).

WHY CHOOSE THIS?

A generator for free: decode a random code.

WHY NOT THIS? (TRADEOFFS)

Samples are blurrier than real digits; sharper generators come in Week 20.

5. Walking between two digits

DEFINITION — start here, no prior knowledge assumed

Seven evenly spaced codes between two test images' codes, decoded, for 200 random pairs; and the straight pixel average for comparison.

REAL-WORLD SCENARIO

Morphing one face into another, frame by frame.

ACTUAL OUTPUT

```
  plain autoencoder   : the 7 in-between images, distance to the nearest real image per pixel:
      median 0.1644
  VAE                 : the 7 in-between images, distance to the nearest real image per pixel:
      median 0.1588
  pixels (no model)   : the 7 in-between images, distance to the nearest real image per pixel:
      median 0.1812
Actual output from this lesson's run.
```

What it means

Pixel averaging gives double exposures (figure, top row) and is furthest from real digits (0.1812). Both models give in-between images that look like plausible digits; the VAE's are a little closer (0.1588 against 0.1644). The gap between the two models is small here: codes between two real codes are less often in a hole than codes drawn at random.

WHY CHOOSE THIS?

Smooth, plausible in-betweens.

WHY NOT THIS? (TRADEOFFS)

A small difference from the plain autoencoder on this measure.

6. Beta: reconstruction against a well-filled code space

DEFINITION — start here, no prior knowledge assumed

16-code VAEs with β = 0.5, 1 and 4, 20 epochs each.

REAL-WORLD SCENARIO

A librarian told to keep shelves tidier and tidier, even if some books end up in roughly the right place.

ACTUAL OUTPUT

```
   beta  reconstruction      KL  active dims   samples: median distance to real
    0.5           83.15   29.50           16                              0.1907
    1.0           87.80   22.79           16                              0.1862
    4.0          116.77    9.38           14                              0.1695
Actual output from this lesson's run.
```

What it means

Raising β trades reconstruction for a smaller KL: from 83.15 and 29.50 at β = 0.5 to 116.77 and 9.38 at β = 4, where 14 of 16 dimensions stay in use. The samples' distance to real images falls to 0.1695, below even real images' own 0.1743, which only shows how this score rewards blur: β = 4 images are vaguer, closer to the average of many digits. A sharp, varied generator needs a better test than nearest-image distance.

WHY CHOOSE THIS?

One knob between faithful codes and an evenly filled space.

WHY NOT THIS? (TRADEOFFS)

Large β blurs; the right value depends on the use.

Interview preparation — technical questions

Q: Why can a VAE generate and a plain autoencoder not?

A: Its codes are pulled to N(0, 1), so random codes decode sensibly: 0.1857 against 0.2048.

Q: What does beta control?

A: The weight on the KL term: $\beta = 4$ cut the KL from 22.79 to 9.38 and raised reconstruction from 87.80 to 116.77.

Q: How would you visualise a VAE?

A: Train with 2 code numbers and decode a grid; here digits form regions that blend smoothly.

Q: How do you judge generated images?

A: Carefully: nearest-image distance rewarded blur here (0.1695 below real images' 0.1743).

Q: Does the VAE reconstruct as well as an autoencoder?

A: Nearly: 0.01236 against 0.01212 per pixel.

Interview preparation — theoretical questions

Q: Why are VAE samples blurry?

A: Partly because the KL term limits how much detail each code can carry, so the decoder hedges between possibilities.

Q: What is an 'active' dimension?

A: One whose KL stays above about 0.1 nats; the others match N(0, 1) and carry nothing.

Q: Why interpolate in code space, not pixels?

A: Pixel averages are double exposures (0.1812); decoded code averages look like digits.

Q: What does the ELBO trade off?

A: Fitting the data (reconstruction) against keeping codes simple (KL).

Key takeaway and what's next

> **KEY TAKEAWAY**
> Trained properly, the VAE keeps almost all of the autoencoder's reconstruction quality and adds a code space without gaps: it can be sampled, walked and drawn. Its weaknesses are measured too: samples are blurry, two dimensions squash digits together, and the obvious closeness score flatters blur. Those weaknesses are what Week 20's generative adversarial networks aim to overcome.

- VAE 0.01236, plain autoencoder 0.01212 per pixel; test loss 109.11 nats.
- Samples: VAE 0.1857, plain autoencoder 0.2048, real images 0.1743 from the nearest real image.
- 2-code map: kNN 0.5291 on just two numbers.
- Beta 4: KL 9.38, reconstruction 116.77; blurrier samples scored 0.1695.

Suggested watch order, if starting from zero

- 1. Variational Autoencoders simplified (Prof. Ryan Ahmed) — why a VAE's latent space is organised.
- 2. Variational Autoencoder - VISUALLY EXPLAINED! (Kapil Sachdeva) — latent variables and amortised inference, visually.

Day 131 preview

- **Contrastive learning**: pulling two views of the same image together and pushing different images apart.